NISHANTHAN V
241801190

EXPERIMENT 5 – INFORMATION EXTRACTION USING spaCy

Learning Objectives

The objective of this experiment is to understand and implement information extraction techniques on unstructured customer review text using the spaCy Natural Language Processing library. The experiment involves loading and cleaning the Amazon Fine Food Reviews dataset, performing text preprocessing, extracting tokens and lemmas, identifying Part-of-Speech (POS) tags, performing Named Entity Recognition (NER), and organizing the extracted information into structured formats for further customer feedback analysis.

Libraries Used / Purpose

- **Pandas** – Used to load, clean, manipulate, and organize the customer review dataset using DataFrames.
- **spaCy** – Used for NLP processing, tokenization, lemmatization, POS tagging, and Named Entity Recognition.
- **Collections.Counter** – Used to count the frequency of different POS categories.
- **Google Colab Files** – Used to upload and access the Amazon Fine Food Reviews dataset.

Program Code

```
!pip install -q spacy
!python -m spacy download en_core_web_sm

import pandas as pd import
spacy from collections import
Counter from google.colab
import files

nlp = spacy.load("en_core_web_sm")

print("spaCy version:", spacy.__version__) print("spaCy
model loaded successfully")

uploaded = files.upload()

filename = list(uploaded.keys())[0] df
= pd.read_csv(filename)
```

```
print("\n--- First 5 Records ---") display(df.head())

print("\n--- Dataset Shape ---") print(df.shape)

print("\n--- Column Names ---") print(df.columns.tolist())

reviews = df[['Text']].copy()

reviews.dropna(subset=['Text'], inplace=True)
reviews.drop_duplicates(subset=['Text'], inplace=True) reviews
= reviews.head(10000).reset_index(drop=True)

print("\n--- Dataset After Cleaning ---") print("Number
of reviews:", len(reviews))

print("\n--- Missing Values ---") print(reviews.isnull().sum())

print("\n--- Duplicate Reviews ---") print(reviews['Text'].duplicated().sum())

display(reviews.head(10))

print("\n" + "=" * 80)
print("TOKENIZATION, LEMMATIZATION AND POS TAGGING") print("="
* 80)

for i, text in enumerate(reviews['Text'].head(5), start=1):

    doc = nlp(text)

    print("\nREVIEW", i)
print("-" * 80)

    print("\nOriginal Review:")
print(text)

    print("\nTokens:")
    print([token.text for token in doc])

    print("\nLemmas:")
print([token.lemma_ for token in doc])

    print("\nPart-of-Speech Tags:")
    print([(token.text, token.pos_) for token in doc])
```

```
    nouns = [token.text for token in doc if token.pos_ in ['NOUN', 'PROPN']]
verbs = [token.text for token in doc if token.pos_ == 'VERB']     adjectives
= [token.text for token in doc if token.pos_ == 'ADJ']     adverbs =
[token.text for token in doc if token.pos_ == 'ADV']

    print("\nNouns:", nouns)
print("Verbs:", verbs)
print("Adjectives:", adjectives)
print("Adverbs:", adverbs)

print("\n" + "=" * 80)
print("FREQUENCY OF MAJOR POS CATEGORIES") print("="
* 80)

pos_counter = Counter()

for doc in nlp.pipe(reviews['Text'], batch_size=100):

    for token in doc:

        if token.pos_ in ['NOUN', 'PROPN']:
            pos_counter['NOUN'] += 1

        elif token.pos_ == 'VERB':
pos_counter['VERB'] += 1

        elif token.pos_ == 'ADJ':
            pos_counter['ADJECTIVE'] += 1

        elif token.pos_ == 'ADV':
            pos_counter['ADVERB'] += 1

pos_frequency = pd.DataFrame(
pos_counter.items(),     columns=['POS
Category', 'Frequency']
).sort_values(
    by='Frequency',
ascending=False
).reset_index(drop=True)

display(pos_frequency)

print("\n" + "=" * 80)
```

```
print("NAMED ENTITY RECOGNITION") print("="
* 80)

required_entities = {
    'PERSON',
    'ORG',
    'DATE',
    'MONEY',
    'QUANTITY',
    'PRODUCT'
}

entity_records = []

for review_no, text in enumerate(reviews['Text'], start=1):

    doc = nlp(text)

    for ent in doc.ents:

        if ent.label_ in required_entities:

            entity_records.append({
                'Review_No': review_no,
                'Review_Text': text,
                'Entity': ent.text,
                'Entity_Label': ent.label_
            })

entity_df = pd.DataFrame(entity_records)

if not entity_df.empty:
    display(entity_df.head(50)) else:     print("No entities from the
selected categories were recognized.")

if not entity_df.empty:

    print("\n--- Named Entity Frequency ---")

    entity_frequency = (
entity_df['Entity_Label']
        .value_counts()
        .reset_index()
    )
```

```
    entity_frequency.columns = [
        'Entity Label',
        'Frequency'
    ]

    display(entity_frequency)

    print("\n--- Unique Extracted Entities ---")

    unique_entities = (
entity_df[
            ['Entity', 'Entity_Label']
        ]
        .drop_duplicates()
        .reset_index(drop=True)
    )

    display(unique_entities.head(50))

print("\n" + "=" * 80)
print("INFORMATION EXTRACTION FUNCTION")
print("=" * 80)

def extract_information(review):

    doc = nlp(review)

    tokens = [token.text for token in doc]

    lemmas = [token.lemma_ for token in doc]

    pos_tags = [
(token.text, token.pos_)
for token in doc
    ]

    named_entities = [
ent.text
        for ent in doc.ents
    ]
```

```
    entity_labels = [
ent.label_         for ent
in doc.ents
    ]

    return {
        'Tokens': tokens,
        'Lemmas': lemmas,
        'POS Tags': pos_tags,
        'Named Entities': named_entities,
        'Entity Labels': entity_labels
    }

new_reviews = [
    "I bought this coffee from Amazon for $25 and received it on January 10, 2026.",
    "The Starbucks coffee tastes excellent and the package contains 500 grams.",
    "John recommended this product because the company delivered it to New York in two days."
]

results = []

for i, review in enumerate(new_reviews, start=1):

    information = extract_information(review)

    results.append({
'Review_No': i,
        'Review': review,
        'Tokens': information['Tokens'],
        'Lemmas': information['Lemmas'],
        'POS_Tags': information['POS Tags'],
        'Named_Entities': information['Named Entities'],
        'Entity_Labels': information['Entity Labels']
    })

results_df = pd.DataFrame(results)

print("\n--- Information Extracted from New Reviews ---")
display(results_df)

new_entity_records = []

for review_no, review in enumerate(new_reviews, start=1):
```

```
    doc = nlp(review)

    for ent in doc.ents:

        new_entity_records.append({
            'Review_No': review_no,
            'Entity': ent.text,
            'Entity_Label': ent.label_
        })

new_entity_df = pd.DataFrame(new_entity_records)

print("\n--- Entities Extracted from New Reviews ---")

if not new_entity_df.empty:
display(new_entity_df) else:
    print("No named entities were recognized.")
```

Output

```
--- Dataset After Cleaning ---
Number of reviews: 10000

--- Missing Values ---
Text    0 dtype: int64

--- Duplicate Reviews ---
0
```

Learning Outcome

After completing this experiment, students will be able to preprocess and clean customer review data using Pandas, apply spaCy NLP pipelines to unstructured text, perform tokenization and lemmatization, identify and analyze Part-of-Speech tags, calculate the frequency of major POS categories, perform Named Entity Recognition, extract meaningful entities such as PERSON, ORG, DATE, MONEY, QUANTITY, and PRODUCT when recognized by the selected spaCy model, develop a reusable information extraction function, and organize extracted linguistic information into structured formats for further customer feedback analysis.